

STORED PROCEDURES

Scenario 1: Write a stored procedure ProcessMonthlyInterest that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
AS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance * 0.01)
    WHERE AccountType = 'Savings';

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Monthly interest processed successfully.');
```



```
END;
/

BEGIN
    ProcessMonthlyInterest;
END;
/
```

NavigatorFiles

My Schema

Tables

Search

Refresh

More

ACCOUNTS

CUSTOMERS

EMPLOYEES

LOANS

TRANSACTIONS

[SQL Worksheet]*

Run

Format

Find

Save

Print

Aa

Close

```
1 CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
2 AS
3 BEGIN
4     UPDATE Accounts
5     SET Balance = Balance + (Balance * 0.01)
6     WHERE AccountType = 'Savings';
7
8     COMMIT;
9
10    DBMS_OUTPUT.PUT_LINE('Monthly interest processed successfully.');
```

Query resultScript outputDBMS outputExplain PlanSQL history

CloseDownload

SQL> CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
AS
BEGIN
 UPDATE Accounts...
[Show more...](#)

Procedure PROCESSMONTHLYINTEREST compiled

Elapsed: 00:00:00.029



FreeSQL

>_ Worksheet

SQL Tutorials

Navigator

Files

My Schema

Tables

Search



ACCOUNTS

CUSTOMERS

EMPLOYEES

LOANS

TRANSACTIONS

[SQL Worksheet]*



A

```
1 BEGIN
2   ProcessMonthlyInterest;
3 END;
4
```

Query result

Script output

DBMS output

Explain



Procedure PROCESSMONTHLYINTEREST compiled

Elapsed: 00:00:00.029

```
SQL> BEGIN
      ProcessMonthlyInterest;
      END;
```

Monthly interest processed successfully.

Scenario 2: Write a stored procedure UpdateEmployeeBonus that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(  
    p_Department VARCHAR2,  
    p_BonusPercent NUMBER  
)  
AS  
BEGIN  
    UPDATE Employees  
    SET Salary = Salary + (Salary * p_BonusPercent / 100)  
    WHERE Department = p_Department;  
  
    COMMIT;  
  
    DBMS_OUTPUT.PUT_LINE('Employee bonus updated successfully.');
```

END;
/

BEGIN
 UpdateEmployeeBonus('IT',10);
END;
/



FreeSQL

>_ Worksheet

SQL Tutorials

Navigator

Files

My Schema ▾

Tables ▾

Q Search



▶ ACCOUNTS

▶ CUSTOMERS

▶ EMPLOYEES

▶ LOANS

▶ TRANSACTIONS

[SQL Worksheet]* ▾



Aa ▾



```
1 CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(  
2     p_Department VARCHAR2,  
3     p_BonusPercent NUMBER  
4 )  
5 AS  
6 BEGIN  
7     UPDATE Employees  
8     SET Salary = Salary + (Salary * p_BonusPercent / 100)  
9     WHERE Department = p_Department;  
10  
11     COMMIT;  
12  
13     DBMS_OUTPUT.PUT_LINE('Employee bonus updated successfully.');
```

Query result

Script output

DBMS output

Explain Plan

SQL history



```
SQL> CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(  
      p_Department VARCHAR2,  
      p_BonusPercent NUMBER  
    )...  
Show more...
```

Procedure UPDATEEMPLOYEEBONUS compiled

Elapsed: 00:00:00.020

NavigatorFiles

My Schema

Tables

Search

ACCOUNTS

CUSTOMERS

EMPLOYEES

LOANS

TRANSACTIONS

[SQL Worksheet]*

1

BEGIN

2

UpdateEmployeeBonus('IT',10);

3

END;

4

/

Query result

Script output

DBMS output

Explain Plan

SQL> BEGIN
UpdateEmployeeBonus('IT',10);
END;

Employee bonus updated successfully.

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.020

Scenario 3: Write a stored procedure TransferFunds that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

```
CREATE OR REPLACE PROCEDURE TransferFunds(  
    p_FromAccount NUMBER,  
    p_ToAccount NUMBER,  
    p_Amount NUMBER  
)  
AS  
    v_Balance NUMBER;  
BEGIN  
    SELECT Balance  
    INTO v_Balance  
    FROM Accounts  
    WHERE AccountID = p_FromAccount;  
  
    IF v_Balance >= p_Amount THEN  
  
        UPDATE Accounts  
        SET Balance = Balance - p_Amount  
        WHERE AccountID = p_FromAccount;  
  
        UPDATE Accounts  
        SET Balance = Balance + p_Amount  
        WHERE AccountID = p_ToAccount;  
  
        COMMIT;  
  
        DBMS_OUTPUT.PUT_LINE('Fund transfer completed successfully.');  
    ELSE  
  
        DBMS_OUTPUT.PUT_LINE('Insufficient balance.');  
    END IF;  
END;  
/
```

The screenshot shows a SQL IDE interface. On the left is a 'Navigator' pane with 'My Schema' and 'Tables' sections. Below them is a search bar and a list of database objects: ACCOUNTS, CUSTOMERS, EMPLOYEES, LOANS, and TRANSACTIONS. The main area is titled '[SQL Worksheet]*' and contains a SQL script for creating a procedure named 'TransferFunds'. The script defines parameters for source account, target account, and amount, then selects the balance from the source account and updates it based on the transfer amount. Below the script editor are tabs for 'Query result', 'Script output', 'DBMS output', and 'Explain Plan'. The 'Script output' tab is active, showing the execution log: 'SQL> CREATE OR REPLACE PROCEDURE TransferFunds(...)', 'Procedure TRANSFERFUNDS compiled', and 'Elapsed: 00:00:00.018'.

```
1 CREATE OR REPLACE PROCEDURE TransferFunds(  
2     p_FromAccount NUMBER,  
3     p_ToAccount NUMBER,  
4     p_Amount NUMBER  
5 )  
6 AS  
7     v_Balance NUMBER;  
8 BEGIN  
9     SELECT Balance  
10    INTO v_Balance  
11   FROM Accounts  
12  WHERE AccountID = p_FromAccount;  
13  
14     IF v_Balance >= p_Amount THEN  
15  
16         UPDATE Accounts  
17
```

Query result **Script output** DBMS output Explain Plan

SQL> CREATE OR REPLACE PROCEDURE TransferFunds(
 p_FromAccount NUMBER,
 p_ToAccount NUMBER,
 p_Amount NUMBER...
[Show more...](#)

Procedure TRANSFERFUNDS compiled

Elapsed: 00:00:00.018

Successful:

```
BEGIN  
    TransferFunds(102,105,1000);  
END;  
/
```


FreeSQL

>_ Worksheet

SQL Tutorials

NavigatorFiles

My Schema

Tables

Search

ACCOUNTS

CUSTOMERS

EMPLOYEES

LOANS

TRANSACTIONS

[SQL Worksheet]*

▶▶▶▶▶▶Aa

1 BEGIN

2 TransferFunds(102,105,1000);

3 END;

4

Query resultScript outputDBMS outputExplain PlanSQ

SQL> BEGIN

TransferFunds(102,105,1000);

END;

Fund transfer completed successfully.

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.006

Insufficient Balance:

```
BEGIN
  TransferFunds(101,105,50000);
END;
/
```

Navigator

Files

My Schema ▼

Tables ▼

Search



▶ ACCOUNTS

▶ CUSTOMERS

▶ EMPLOYEES

▶ LOANS

▶ TRANSACTIONS

[SQL Worksheet]* ▼



Aa ▼

```
1 BEGIN
2     TransferFunds(101,105,50000);
3 END;
4 /
```

Query result

Script output

DBMS output

Explain Plan



```
SQL> BEGIN
      TransferFunds(101,105,50000);
      END;
```

Insufficient balance.

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.012